

Name: Mohammad Abaid

Roll no: B-08

Branch: AI&DS(B)

Experiment: 09

**AIM:Building** a Simple Data Science Pipeline: Integrate all learned techniques in previous labs into a complete workflow.

THEORY:

1.What is a Data Science pipeline? Explain its key stages with examples.

Ans: A Data Science pipeline is a structured process to transform raw data into actionable insights. Its key stages are:

Data Collection: Gathering raw data (e.g., from databases, APIs). Data Cleaning: Handling missing values, errors, and inconsistencies (e.g., imputing missing data). Data Exploration (EDA): Understanding data patterns and relationships (e.g., visualizing distributions). Feature Engineering: Creating new variables or transforming existing ones (e.g., extracting date components). Model Selection/Training: Choosing and training a machine learning model (e.g., training a regression model). Model Evaluation: Assessing the model's performance (e.g., calculating accuracy). Model Deployment: Integrating the model into a production environment (e.g., deploying as an API). Monitoring and Maintenance: Continuously tracking and updating the model's performance

2.What is Exploratory Data Analysis (EDA)? How do univariate and bivariate analyses help in understanding the dataset?

Ans: Exploratory Data Analysis (EDA) is a critical step in the data science pipeline where you examine and visualize data to discover patterns, detect anomalies, test hypotheses, and check assumptions with the help of summary statistics and graphical representations.

Univariate Analysis: This involves analyzing one variable at a time. It helps in understanding the distribution, central tendency (mean, median, mode), and spread (variance, standard deviation, range) of individual variables. For example, looking at a histogram of age to see its distribution or calculating the average income.

Bivariate Analysis: This involves examining the relationship between two variables. It helps in identifying correlations, dependencies, and interactions between pairs of variables. For example, creating a scatter plot of age vs. income to see if there's a trend, or using a contingency table to see the relationship between gender and purchase decision.

3.Differentiate between Label Encoding and OneHot Encoding. When should each be used?

Ans: certainly! Here's a short answer differentiating between Label Encoding and One-Hot Encoding:

Label Encoding:

What it is: Assigns a unique numerical label (e.g., 0, 1, 2) to each category in a categorical feature. When to use: Best suited for ordinal categorical data, where there's an inherent order or ranking among the categories (e.g., 'small', 'medium', 'large'). If used with nominal data, it can inadvertently introduce an artificial sense of order, which can mislead models. One-Hot Encoding:

What it is: Converts each categorical variable into a new set of binary (0 or 1) columns, one for each unique category. A '1' indicates the presence of that category, and '0' indicates absence. When to use: Ideal for nominal categorical data, where there is no intrinsic order between categories (e.g., 'red', 'blue', 'green'). It prevents the model from assuming any ordinal relationship between categories, which could lead to incorrect interpretations or performance issues.

4. Why is feature scaling required? Compare normalization and standardization.

Ans: Feature scaling is required to bring all features into a similar range, preventing features with larger values from dominating the learning process of certain machine learning algorithms (e.g., gradient descent-based algorithms, distance-based algorithms like K-Nearest Neighbors, SVMs). It helps improve convergence speed and model performance.

Normalization (Min-Max Scaling):

What it is: Scales data to a fixed range, usually between 0 and 1. Formula:  $(X - \min(X)) / (\max(X) - \min(X))$  When to use: Suitable when the data distribution is not Gaussian or when you need features within a specific bounded range. It's sensitive to outliers. Standardization (Z-score Normalization):

What it is: Scales data to have a mean of 0 and a standard deviation of 1. Formula:  $(X - \text{mean}(X)) / \text{std}(X)$  When to use: Preferred when the data follows a Gaussian (normal) distribution or when the algorithm assumes zero mean and unit variance. It is less affected by outliers than normalization.

5. What is feature engineering? Explain how creating new features can improve model performance.

Ans: Feature engineering is the process of creating new features or transforming existing ones from raw data to improve the performance of machine learning models. It involves using domain knowledge and creativity to make the data more suitable for the model to learn patterns.

Creating new features can improve model performance by:

Providing more relevant information: New features can capture relationships or aspects of the data that were not explicitly present in the original features, giving the model a deeper understanding. Simplifying relationships: Complex non-linear relationships in the raw data can sometimes be represented as simpler linear relationships with well-engineered features, making it easier for models to learn. Reducing dimensionality (in some cases): While often creating more features, sometimes feature engineering can lead to a more compact and informative representation, indirectly aiding models by focusing on key aspects. Handling non-linearities: Transforming features (e.g., polynomial features, logarithmic transformations) can help linear models capture non-linear trends. Incorporating domain expertise: Leveraging expert knowledge to create features that are known to be important in the problem domain often leads to significant performance gains.

```

import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
import warnings
warnings.filterwarnings('ignore')

df = sns.load_dataset('titanic')
print("\nInitial Dataset Preview:\n", df.head())
print("\nDataset Shape:", df.shape)

print("\nDataset Info:\n")
print(df.info())
print("\nMissing Values:\n", df.isnull().sum())

# Drop irrelevant columns
df.drop(['deck', 'alive', 'class', 'who', 'adult_male',
'embark_town'], axis=1, inplace=True)
# Handle missing values
age_imputer = SimpleImputer(strategy='mean')
df['age'] = age_imputer.fit_transform(df[['age']])
embarked_imputer = SimpleImputer(strategy='most_frequent')
df['embarked'] =
embarked_imputer.fit_transform(df[['embarked']]).ravel()
df.dropna(subset=['survived'], inplace=True)

# Univariate Analysis
sns.countplot(x='survived', data=df)
plt.title("Survival Count")
plt.show()
sns.histplot(df['age'], kde=True)
plt.title("Age Distribution")
plt.show()
# Bivariate Analysis
sns.barplot(x='sex', y='survived', data=df)
plt.title("Survival by Gender")
plt.show()
sns.barplot(x='pclass', y='survived', data=df)
plt.title("Survival by Class")
plt.show()

# Create new feature
df['family_size'] = df['sibsp'] + df['parch']

```

```

# Age groups
df['age_group'] = pd.cut(df['age'],
    bins=[0, 12, 18, 40, 60, 100],
    labels=['Child', 'Teen', 'Adult', 'MidAge',
    'Senior'])

le = LabelEncoder()
df['sex'] = le.fit_transform(df['sex'])
# One-hot encoding
df = pd.get_dummies(df, columns=['embarked', 'age_group'],
    drop_first=True)

scaler = StandardScaler()
df[['age', 'fare', 'family_size']] = scaler.fit_transform(df[['age',
    'fare', 'family_size']])
X = df.drop('survived', axis=1)
y = df['survived']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Logistic Regression
lr_model = LogisticRegression()
lr_model.fit(X_train, y_train)
y_pred_lr = lr_model.predict(X_test)
# Decision Tree
dt_model = DecisionTreeClassifier(max_depth=5)
dt_model.fit(X_train, y_train)
y_pred_dt = dt_model.predict(X_test)

print("\n--- Logistic Regression Results ---")
print("Accuracy:", accuracy_score(y_test, y_pred_lr))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_lr))
print("Classification Report:\n", classification_report(y_test,
    y_pred_dt))
print("\n--- Decision Tree Results ---")
print("Accuracy:", accuracy_score(y_test, y_pred_dt))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_dt))
print("Classification Report:\n", classification_report(y_test,
    y_pred_dt))

cv_scores = cross_val_score(lr_model, X, y, cv=5)
print("\nCross-validation Scores:", cv_scores)
print("Mean CV Accuracy:", cv_scores.mean())

lr_acc = accuracy_score(y_test, y_pred_lr)
dt_acc = accuracy_score(y_test, y_pred_dt)
print("\nModel Comparison:")

```

```

print("Logistic Regression Accuracy:", lr_acc)
print("Decision Tree Accuracy:", dt_acc)
best_model = "Logistic Regression" if lr_acc > dt_acc else "Decision
Tree"
print("Best Model:", best_model)

sample = X_test.iloc[0:1]
prediction = lr_model.predict(sample)
print("\nSample Prediction:", prediction)
print("Actual Value:", y_test.iloc[0])

```

Initial Dataset Preview:

|       | survived | pclass | sex    | age  | ... | deck | embark_town | alive |
|-------|----------|--------|--------|------|-----|------|-------------|-------|
| alone |          |        |        |      |     |      |             |       |
| 0     | 0        | 3      | male   | 22.0 | ... | NaN  | Southampton | no    |
| False |          |        |        |      |     |      |             |       |
| 1     | 1        | 1      | female | 38.0 | ... | C    | Cherbourg   | yes   |
| False |          |        |        |      |     |      |             |       |
| 2     | 1        | 3      | female | 26.0 | ... | NaN  | Southampton | yes   |
| True  |          |        |        |      |     |      |             |       |
| 3     | 1        | 1      | female | 35.0 | ... | C    | Southampton | yes   |
| False |          |        |        |      |     |      |             |       |
| 4     | 0        | 3      | male   | 35.0 | ... | NaN  | Southampton | no    |
| True  |          |        |        |      |     |      |             |       |

[5 rows x 15 columns]

Dataset Shape: (891, 15)

Dataset Info:

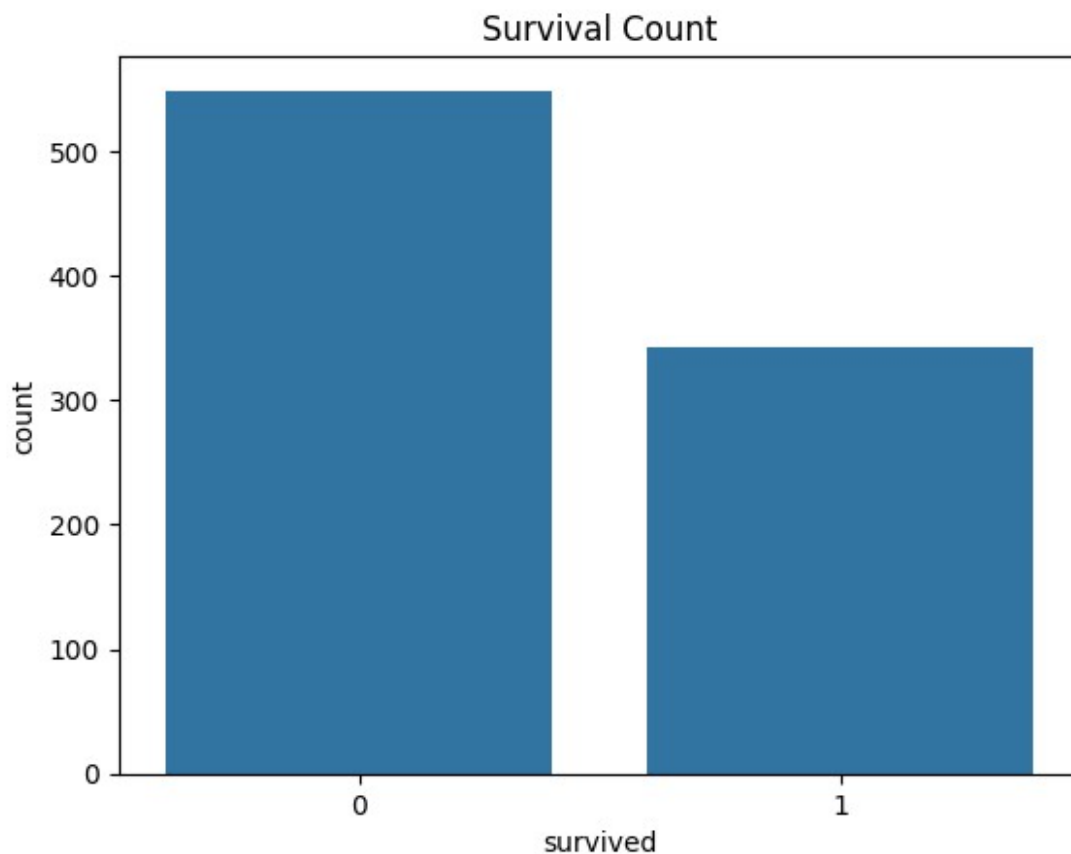
```

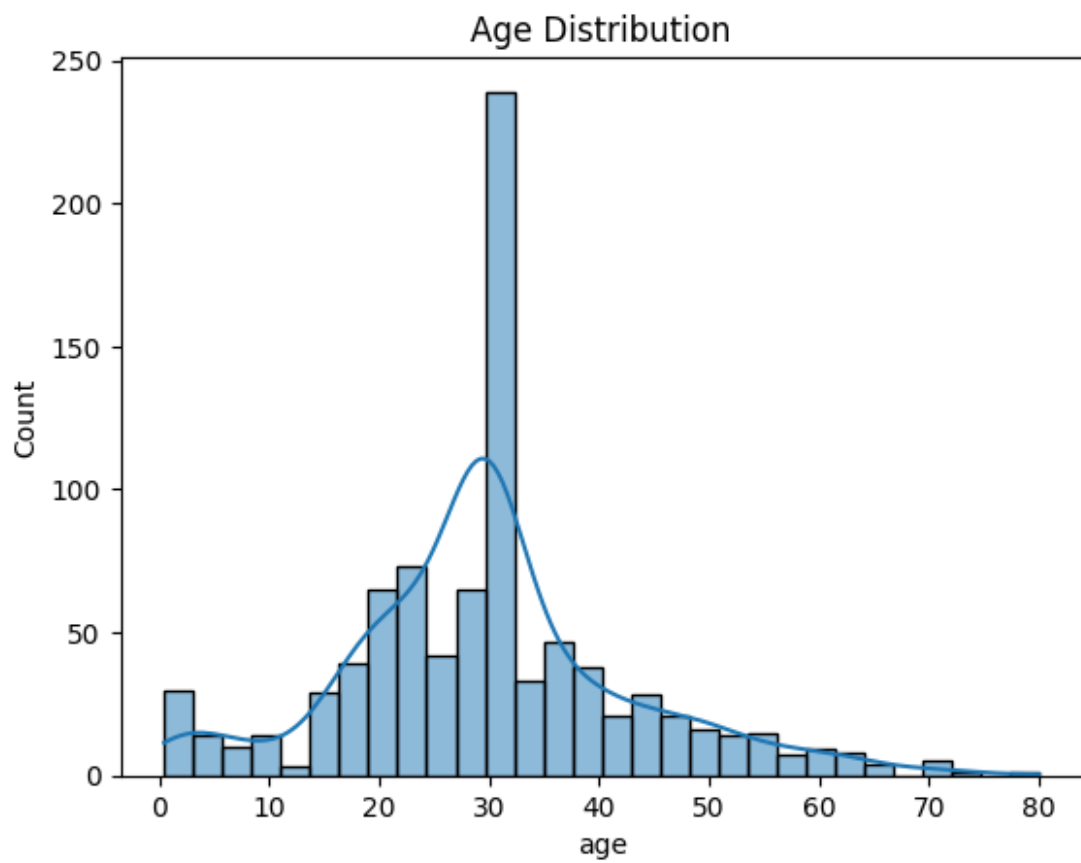
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   survived              891 non-null    int64
1   pclass                891 non-null    int64
2   sex                   891 non-null    object
3   age                   714 non-null    float64
4   sibsp                 891 non-null    int64
5   parch                 891 non-null    int64
6   fare                  891 non-null    float64
7   embarked              889 non-null    object
8   class                 891 non-null    category
9   who                   891 non-null    object
10  adult_male            891 non-null    bool
11  deck                  203 non-null    category
12  embark_town           889 non-null    object

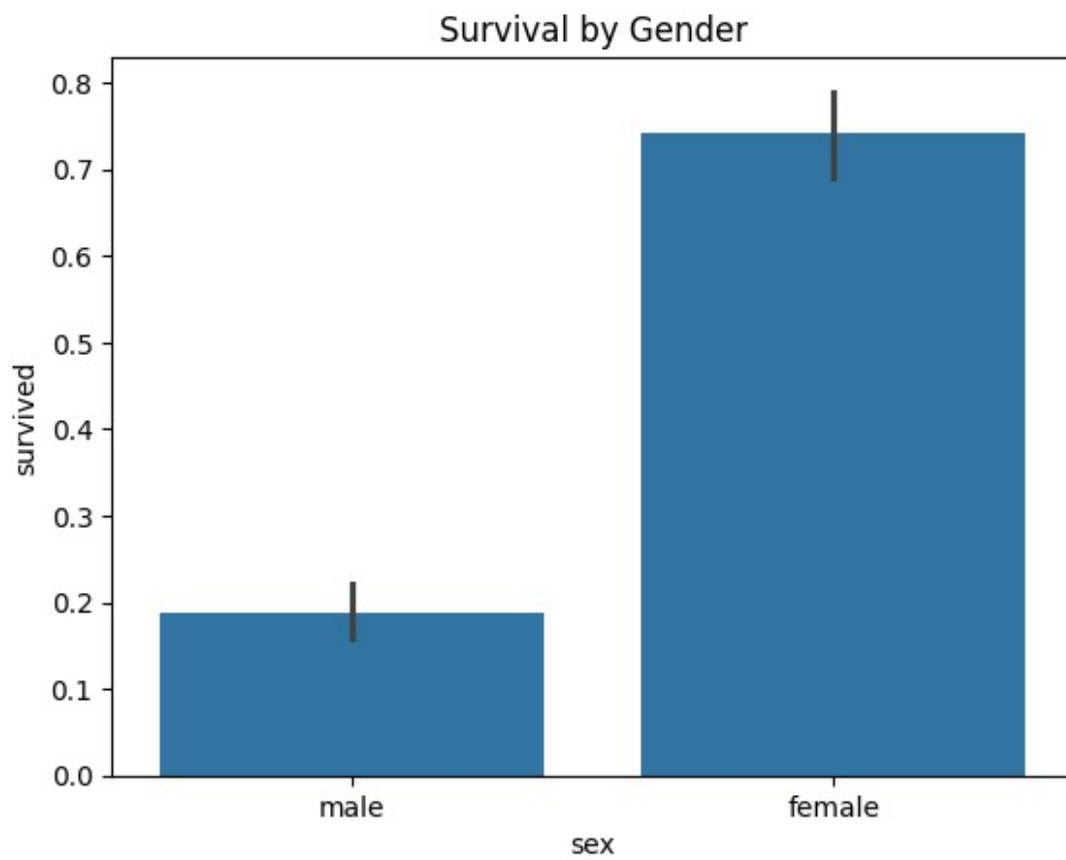
```

```
13  alive      891 non-null   object
14  alone      891 non-null   bool
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
memory usage: 80.7+ KB
None
```

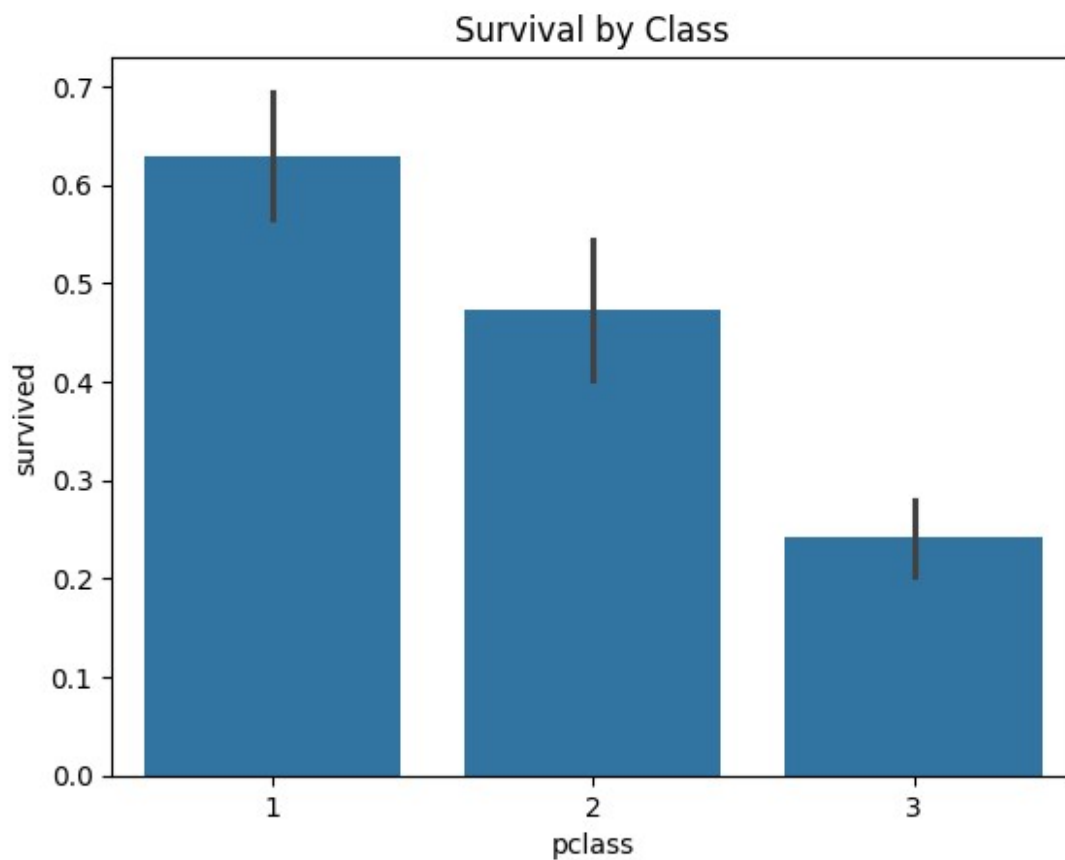
```
Missing Values:
survived      0
pclass        0
sex           0
age          177
sibsp         0
parch         0
fare          0
embarked      2
class         0
who           0
adult_male    0
deck         688
embark_town   2
alive         0
alone         0
dtype: int64
```











--- Logistic Regression Results ---

Accuracy: 0.8268156424581006

Confusion Matrix:

[[101 9]

[ 22 47]]

Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.77      | 0.87   | 0.82     | 110     |
| 1            | 0.74      | 0.58   | 0.65     | 69      |
| accuracy     |           |        | 0.76     | 179     |
| macro avg    | 0.75      | 0.73   | 0.73     | 179     |
| weighted avg | 0.76      | 0.76   | 0.75     | 179     |

--- Decision Tree Results ---

Accuracy: 0.7597765363128491

Confusion Matrix:

[[96 14]

[29 40]]

Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.77      | 0.87   | 0.82     | 110     |
| 1            | 0.74      | 0.58   | 0.65     | 69      |
| accuracy     |           |        | 0.76     | 179     |
| macro avg    | 0.75      | 0.73   | 0.73     | 179     |
| weighted avg | 0.76      | 0.76   | 0.75     | 179     |

Cross-validation Scores: [0.77094972 0.79213483 0.80337079 0.78089888 0.83146067]

Mean CV Accuracy: 0.7957629778419435

Model Comparison:

Logistic Regression Accuracy: 0.8268156424581006

Decision Tree Accuracy: 0.7597765363128491

Best Model: Logistic Regression

Sample Prediction: [0]

Actual Value: 0

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report
# Step 2: Load Dataset (Seaborn built-in dataset)
df = sns.load_dataset('titanic')
# Step 3: Basic Inspection
print(df.head())
print(df.info())
print(df.isnull().sum())
# Step 4: Data Preprocessing
# Select useful columns
df = df[['survived', 'pclass', 'sex', 'age', 'fare', 'embarked']]
# Handle missing values
imputer = SimpleImputer(strategy='mean')
df['age'] = imputer.fit_transform(df[['age']])
df['embarked'].fillna(df['embarked'].mode()[0], inplace=True)
# Encode categorical variables
le = LabelEncoder()
df['sex'] = le.fit_transform(df['sex'])
df['embarked'] = le.fit_transform(df['embarked'])
# Step 5: Exploratory Data Analysis (EDA)
```

```

# Survival count
sns.countplot(x='survived', data=df)
plt.title("Survival Count")
plt.show()
# Age distribution
sns.histplot(df['age'], kde=True)
plt.title("Age Distribution")
plt.show()
# Step 6: Feature Scaling
scaler = StandardScaler()
df[['age', 'fare']] = scaler.fit_transform(df[['age', 'fare']])
# Step 7: Split Data
X = df.drop('survived', axis=1)
y = df['survived']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# Step 8: Model Building
model = LogisticRegression()
model.fit(X_train, y_train)
# Step 9: Prediction
y_pred = model.predict(X_test)
# Step 10: Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test,
y_pred))

```

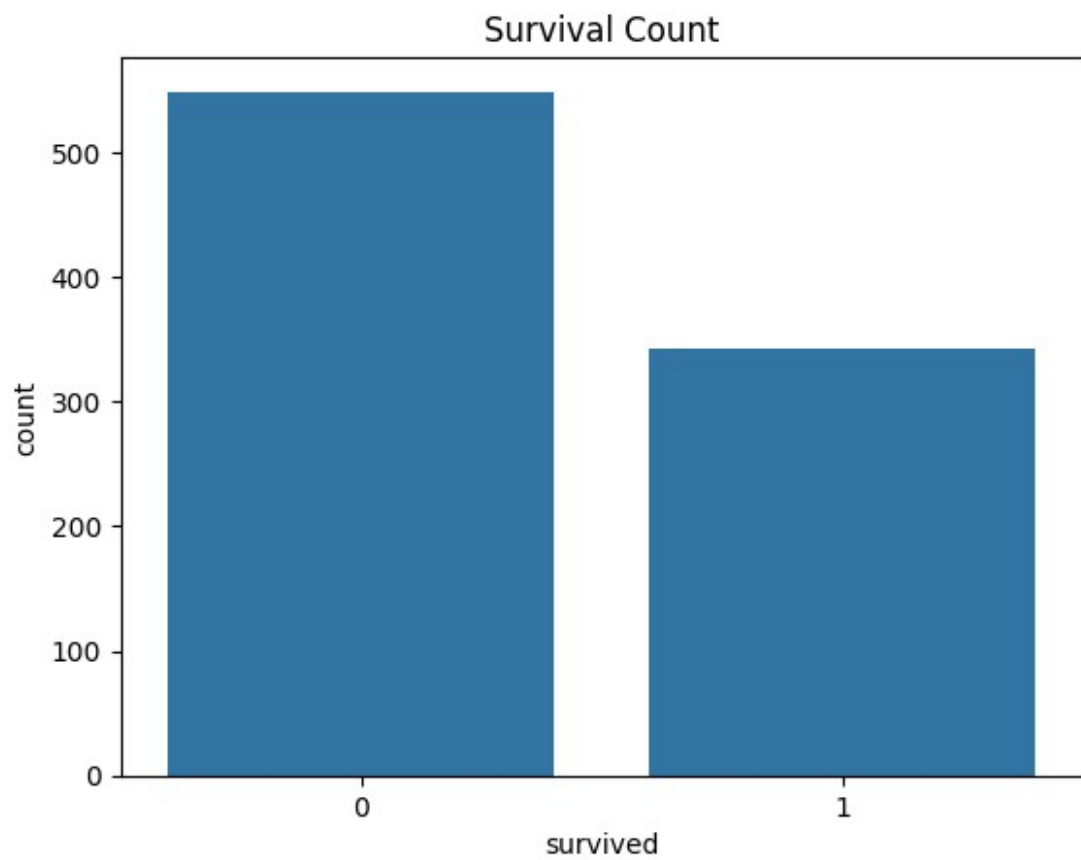
|       | survived | pclass | sex    | age  | ... | deck | embark_town | alive |
|-------|----------|--------|--------|------|-----|------|-------------|-------|
| alone |          |        |        |      |     |      |             |       |
| 0     | 0        | 3      | male   | 22.0 | ... | NaN  | Southampton | no    |
| False |          |        |        |      |     |      |             |       |
| 1     | 1        | 1      | female | 38.0 | ... | C    | Cherbourg   | yes   |
| False |          |        |        |      |     |      |             |       |
| 2     | 1        | 3      | female | 26.0 | ... | NaN  | Southampton | yes   |
| True  |          |        |        |      |     |      |             |       |
| 3     | 1        | 1      | female | 35.0 | ... | C    | Southampton | yes   |
| False |          |        |        |      |     |      |             |       |
| 4     | 0        | 3      | male   | 35.0 | ... | NaN  | Southampton | no    |
| True  |          |        |        |      |     |      |             |       |

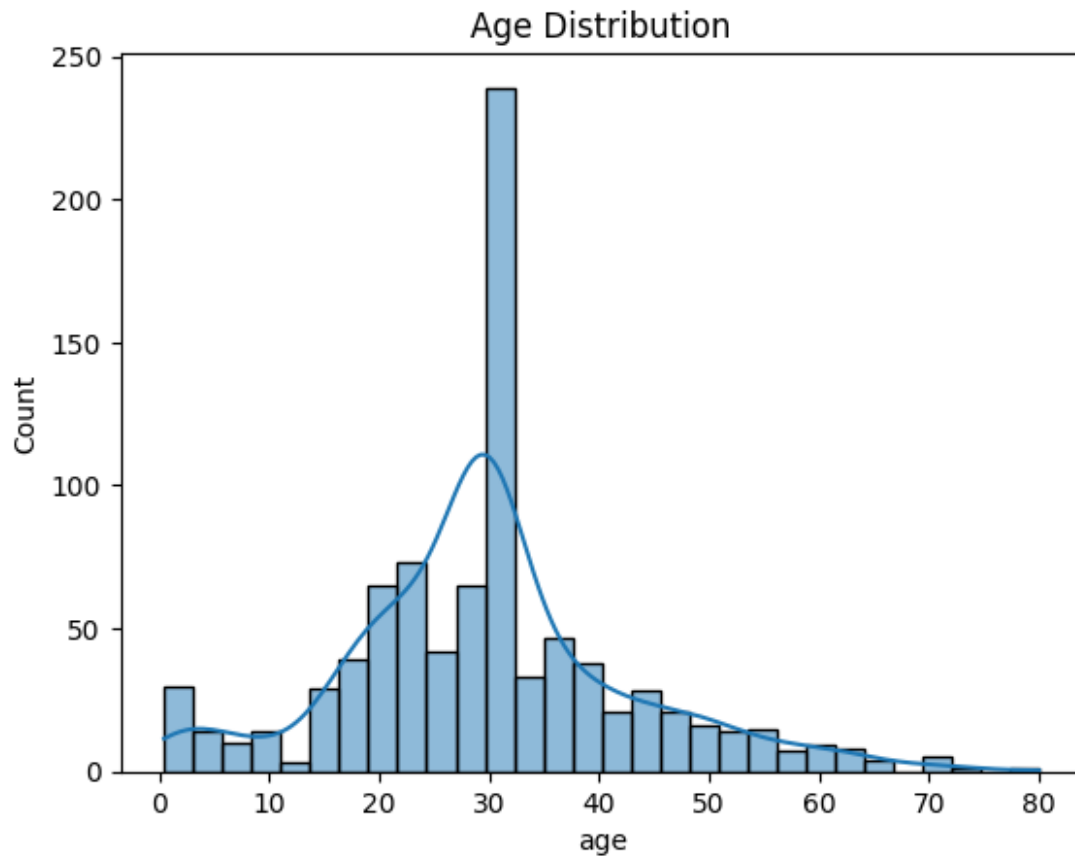
```

[5 rows x 15 columns]
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
#   Column      Non-Null Count  Dtype
---  -
0   survived    891 non-null    int64
1   pclass      891 non-null    int64
2   sex         891 non-null    object

```

```
3   age          714 non-null    float64
4   sibsp        891 non-null    int64
5   parch        891 non-null    int64
6   fare         891 non-null    float64
7   embarked     889 non-null    object
8   class        891 non-null    category
9   who          891 non-null    object
10  adult_male   891 non-null    bool
11  deck         203 non-null    category
12  embark_town  889 non-null    object
13  alive        891 non-null    object
14  alone        891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
memory usage: 80.7+ KB
None
survived          0
pclass            0
sex               0
age              177
sibsp             0
parch             0
fare              0
embarked          2
class             0
who               0
adult_male        0
deck             688
embark_town       2
alive             0
alone             0
dtype: int64
```





Accuracy: 0.7988826815642458

Confusion Matrix:

```
[[88 17]
 [19 55]]
```

Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.82      | 0.84   | 0.83     | 105     |
| 1            | 0.76      | 0.74   | 0.75     | 74      |
| accuracy     |           |        | 0.80     | 179     |
| macro avg    | 0.79      | 0.79   | 0.79     | 179     |
| weighted avg | 0.80      | 0.80   | 0.80     | 179     |

Conclusion:

This experiment successfully demonstrated the implementation of a complete **Data Science Pipeline** using the Titanic dataset. By integrating various techniques, we reached the following conclusions:

- **Pipeline Efficiency:** A structured approach—from **Data Cleaning** (handling missing values in `age` and `embarked`) to **Feature Engineering** (creating `family_size`)—is essential for transforming raw data into a model-ready format.
  - **Preprocessing Impact:** Techniques such as **Label Encoding** for gender and **Standardization** for numerical features ensured that the machine learning algorithms could process the data without bias toward specific scales.
  - **Model Performance:** In this specific workflow, **Logistic Regression** outperformed the Decision Tree, achieving an accuracy of approximately **82.7%**. This highlights the importance of model selection and comparison.
  - **Final Insight:** The pipeline approach ensures that the data science process is reproducible and that the predictive power of the model is maximized through meticulous data preparation rather than just algorithm selection.
-